



Documentación Técnica Completa

Plataforma de Hospedajes — Mérida, Yucatán

Versión: MVP v1.0 • **Fecha:** Junio 2026
• **Dominio:** beel.mx

Email de desarrollo: `dev@beel.mx`

Sitio web: `https://beel.mx`

Sede: Mérida, Yucatán, México

Stack: FastAPI + Next.js 14 + PostgreSQL

Contents

1	La Empresa: Beel	4
1.1	Nombre y Marca	4
1.1.1	Variantes del Logotipo	4
1.2	Misión	5
1.3	Visión	5
1.4	Valores	5
1.5	Modelo de Negocio	5
2	Arquitectura del Sistema	6
2.1	Vista General	6
2.2	Principios de Diseño	6
2.3	Entornos	7
3	Stack Tecnológico	8
3.1	Backend	8
3.2	Frontend	9
3.3	Infraestructura y Servicios Externos	9
4	Estructura del Proyecto	10
5	Backend — Núcleo del Sistema	12
5.1	Punto de Entrada: <code>main.py</code>	12
5.2	Configuración: <code>core/config.py</code>	12
5.2.1	Variables de Entorno Completas	12
5.3	Autenticación: <code>core/auth.py</code>	14
5.4	Cache: <code>core/cache.py</code>	14
6	Módulos del Backend	15
6.1	Módulo: Users	15
6.1.1	Modelo de Datos	15
6.1.2	Endpoints — Users	15
6.2	Módulo: Properties	16
6.2.1	Modelo de Datos	16
6.2.2	Tablas Relacionadas	16
6.2.3	Búsqueda de Propiedades	16
6.2.4	Endpoints — Properties	17
6.3	Módulo: Reservations	17
6.3.1	Máquina de Estados	17

6.3.2	Campos Clave del Modelo	17
6.3.3	Tabla: Availability	17
6.3.4	Endpoints — Reservations	18
6.4	Módulo: Payments	18
6.4.1	Flujo de Pago	18
6.4.2	Endpoints — Payments	19
6.5	Módulo: Messaging	19
6.5.1	Diseño	19
6.5.2	Server-Sent Events (SSE)	19
6.5.3	Endpoints — Messaging	19
6.6	Módulo: Reviews	20
6.6.1	Diseño Bidireccional	20
6.6.2	Endpoints — Reviews	20
6.7	Módulo: Webhooks	20
7	Base de Datos	21
7.1	Extensiones PostgreSQL	21
7.2	Tablas Principales	21
7.3	Migraciones Alembic	22
7.4	Índices Clave	22
8	Frontend	23
8.1	Arquitectura Next.js 14 App Router	23
8.2	Design System — Tokens Beel	23
8.3	Páginas	24
8.4	Componentes	24
8.5	Hooks	24
9	Seguridad	25
9.1	Autenticación y Autorización	25
9.2	Protección de Datos	25
9.3	Validación de Webhooks	25
9.4	Rate Limiting y CORS	25
10	Flujos Principales	26
10.1	Registro y Onboarding	26
10.2	Conversión a Anfitrión	26
10.3	Flujo Completo de Reserva	26
10.4	Flujo de Reseña	27
11	Referencia Completa de API	28
12	Decisiones Técnicas Clave	30
12.1	Anti-doble-booking	30
12.2	Snapshots de Precio	30
12.3	Mensajería Transport-Agnostic	30
12.4	Privacidad de Coordenadas	30
12.5	Clerk como Identity Provider	30

13 Guía de Desarrollo Local	31
13.1 Requisitos Previos	31
13.2 Setup del Backend	31
13.3 Setup del Frontend	31
14 Roadmap	32
14.1 MVP v1 — Estado Actual	32
14.2 Próximas Iteraciones	32
Apéndice A: Catálogo de Amenidades	33
Apéndice B: Tipos de Propiedad	34

Chapter 1

La Empresa: Beel

Identidad de Marca

Beel es el marketplace de hospedajes especializado en Mérida, Yucatán. Conecta a viajeros que buscan alojamiento auténtico con anfitriones locales, priorizando confianza, transparencia y una experiencia 100% adaptada al contexto mexicano.

1.1 Nombre y Marca

El nombre **Beel** es corto, memorable y fácil de pronunciar tanto en español como en inglés. El dominio principal es **beel.mx**, alineado con la identidad local yucateca. La paleta de colores corporativos refleja la naturaleza y cultura de la región:

Color	Hex	Uso
Verde Primario	#1D9E75	CTA, botones, íconos principales
Arena	#F1EFE8	Fondos cálidos, tarjetas
Tierra	#2C2C2A	Textos principales, encabezados
Acento Naranja	#F5A623	Destacados, badges, llamadas

1.1.1 Variantes del Logotipo

El proyecto incluye las siguientes variantes de logotipo:

- `beel_logo_green_sand.png` — Verde sobre arena (uso principal en web)
- `beel_logo_white_green.png` — Blanco sobre verde (fondos oscuros)
- `beel_logo_black_sand.png` — Negro sobre arena (impresión)
- `beel_logo_black_white.png` — Negro sobre blanco (documentos)
- `beel_icon_app.png` — Ícono app móvil

- `beel_symbol_app.png` — Símbolo aislado

1.2 Misión

Misión

Hacer que encontrar y reservar un hospedaje en Mérida sea tan fácil, seguro y transparente como debería ser: sin comisiones abusivas en el lanzamiento, con pagos locales en pesos mexicanos, y poniendo a los anfitriones y huéspedes yucatecos en el centro de cada transacción.

1.3 Visión

Visión

Ser la plataforma de referencia de hospedajes en el Sureste Mexicano para 2027, expandiéndose progresivamente a las ciudades coloniales y destinos costeros de la región (Valladolid, Izamal, Progreso, Holbox), construyendo la comunidad de anfitriones más confiable de México.

1.4 Valores

1. **Confianza:** cada reserva está protegida. El pago al anfitrión se libera 24 h después del check-in.
2. **Transparencia:** precios claros, sin sorpresas. La comisión de plataforma es **0%** en fase de lanzamiento.
3. **Localismo:** pagos en MXN con MercadoPago, soporte en español.
4. **Privacidad:** las coordenadas exactas se ofuscan 150 m en el mapa público.
5. **Calidad:** revisión manual de cada propiedad antes de activarse.

1.5 Modelo de Negocio

Beel opera como marketplace de doble cara (two-sided marketplace):

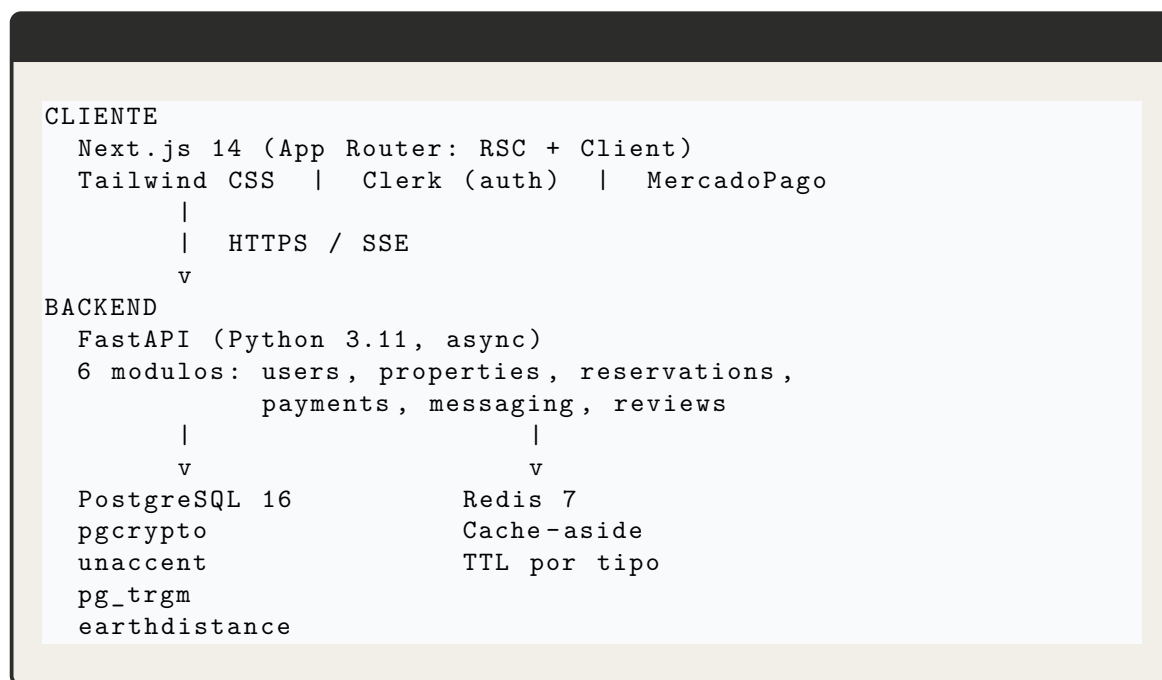
Actor	Propuesta de Valor	Monetización
Huésped	Hospedaje auténtico, pago seguro	Cargo por servicio (futuro)
Anfitrión	Publicar y gestionar reservas	Comisión sobre reserva (futuro)
Beel	Infraestructura, confianza	PLATFORM_FEE = 0% en MVP

Chapter 2

Arquitectura del Sistema

2.1 Vista General

Beel sigue una arquitectura cliente-servidor con separación estricta entre frontend y backend, comunicados vía API REST + Server-Sent Events (SSE) para tiempo real.



2.2 Principios de Diseño

1. **UUIDs como PKs** — seguridad y compatibilidad distribuida.
2. **Soft deletes** — campo `deleted_at`; preserva historial.
3. **Snapshots en reservas** — precio y política se congelan al crear la reserva.
4. **Anti-doble-booking** — `SELECT FOR UPDATE SKIP LOCKED`.
5. **Transport-agnostic messaging** — SSE hoy; migración a WebSocket sin cambiar schema.

6. **Cache-aside con Redis** — TTL diferenciados por tipo de dato.
7. **Rate limiting** — SlowAPI, 60 req/min por defecto.
8. **Ofuscación de coordenadas** — radio aleatorio de 150 m en mapa público.

2.3 Entornos

Variable	Development	Production
ENVIRONMENT	development	production
DEBUG	True	False
Docs UI (/docs)	Habilitada	Deshabilitada
MercadoPago	Sandbox	Producción
Sentry	Opcional	Requerido

Chapter 3

Stack Tecnológico

3.1 Backend

Tecnología	Versión	Rol
Python	3.11+	Lenguaje principal
FastAPI	^0.111	Web framework async
Uvicorn + Standard	^0.30	Servidor ASGI
SQLAlchemy	^2.0	ORM async (<code>Mapped[]</code>)
asyncpg	^0.29	Driver PostgreSQL async
Alembic	^1.13	Migraciones de DB
Pydantic v2	^2.8	Validación y schemas
pydantic-settings	^2.3	Variables de entorno
redis[hiredis]	^5.0	Cache async
python-jose[cryptography]	^3.3	JWT RS256 (Clerk)
httpx	^0.27	HTTP async (JWKS, webhooks)
boto3	^1.34	AWS S3 (fotos)
mercadopago	^2.2	SDK de pagos
sendgrid	^6.11	Email transaccional
sentry-sdk[fastapi]	^2.11	Monitoreo de errores
slowapi	^0.1	Rate limiting

3.2 Frontend

Tecnología	Versión	Rol
Next.js	14.2.3	Framework React (App Router)
React	^18.3	UI library
TypeScript	^5.4	Tipado estático
Tailwind CSS	^3.4	Estilos utility-first
@clerk/nextjs	^5.1	Autenticación
date-fns	^3.6	Manipulación de fechas
react-day-picker	^8.10	Calendario de reservas
lucide-react	^0.383	Íconos SVG
clsx + tailwind-merge	latest	Composición de clases CSS

3.3 Infraestructura y Servicios Externos

Servicio	Función
PostgreSQL 16	Base de datos con extensiones geoespaciales
Redis 7	Cache-aside con TTL por tipo de entidad
AWS S3 + CloudFront	Almacenamiento y CDN de fotos
Clerk	Autenticación: JWT RS256, webhooks, sesiones
MercadoPago	Pagos en MXN: preferencias, webhooks HMAC-SHA256
Google Maps	Geocodificación de direcciones
Sentry	Monitoreo de errores y trazas de rendimiento
Resend / SendGrid	Email transaccional
360dialog	Notificaciones por WhatsApp Business

Chapter 4

Estructura del Proyecto

Listing 4.1: Arbol completo del repositorio Beel

```
1 beel/
2 |-- backend/
3 |   |-- alembic/
4 |   |   |-- alembic.ini
5 |   |   |-- env.py
6 |   |   |-- versions/
7 |   |       |-- 001_initial_extensions_and_functions.py
8 |   |       |-- 002_users_and_auth.py
9 |   |       |-- 003_properties.py
10 |   |       |-- 004_availability_reservations_payments.py
11 |   |       |-- 005_messaging_reviews_notifications.py
12 |   |       |-- 006_seed_amenities.py
13 |   |-- app/
14 |   |   |-- core/
15 |   |   |   |-- analytics.py      # Tracking de eventos fire-and-
16 |   |   |   forget
17 |   |   |   |-- auth.py          # JWT RS256 Clerk + ClerkUser
18 |   |   |   dataclass
19 |   |   |   |-- cache.py         # Redis cache-aside + generadores de
20 |   |   |   claves
21 |   |   |   |-- config.py       # Pydantic Settings (todas las env
22 |   |   |   vars)
23 |   |   |   |-- database.py     # AsyncEngine, SessionLocal,
24 |   |   |   TimestampMixin
25 |   |   |   |-- modules/
26 |   |   |       |-- messaging/  # Conversaciones + SSE
27 |   |   |       |-- payments/   # MercadoPago checkout + webhooks
28 |   |   |       |-- properties/ # Búsqueda, CRUD, fotos, amenidades
29 |   |   |       |-- reservations/ # Reservas, disponibilidad, precios
30 |   |   |       |-- reviews/    # Resenas bidireccionales
31 |   |   |       |-- users/      # Perfiles, roles, conversion a host
32 |   |   |       |-- webhooks/   # Clerk webhook (sync de usuarios)
33 |   |   |       |-- main.py     # FastAPI app, middleware, routers
34 |   |   |-- database/
35 |   |       |-- schema.sql      # Schema SQL completo (referencia)
36 |   |       |-- pyproject.toml  # Dependencias (Poetry)
37 |   |       |-- .env.example
38 |   |
39 |   |-- frontend/
40 |   |   |-- app/                # Next.js App Router
```

```

36 | | | -- layout.tsx          # Root layout + ClerkProvider
37 | | | -- page.tsx           # Homepage (RSC)
38 | | | -- not-found.tsx
39 | | | -- buscar/page.tsx   # Búsqueda + filtros (RSC)
40 | | | -- mensajes/page.tsx # Inbox + chat SSE (Client)
41 | | | -- p/[id]/page.tsx   # Detalle de propiedad (RSC)
42 | | | -- p/[id]/reservar/page.tsx # Checkout (Client)
43 | | | -- reservaciones/page.tsx # Mis viajes (Client)
44 | | -- components/
45 | | | -- AmenityList.tsx   # Amenidades con emojis
46 | | | -- BookingWidget.tsx # Widget de reserva sticky
47 | | | -- Navbar.tsx       # Barra de navegacion Clerk
48 | | | -- PropertyCard.tsx  # Tarjeta + carousel + favoritos
49 | | | -- SearchBar.tsx     # Barra búsqueda
50 | | | -- SearchFilters.tsx # Filtros (tipo, precio...)
51 | | -- hooks/
52 | | | -- useApi.ts         # Fetch autenticado con JWT
53 | | | -- useSSE.ts        # EventSource + backoff
54 | | -- lib/utils.ts        # cn(), formatPrice()...
55 | | -- styles/globals.css   # Variables CSS, clases Beel
56 | | -- types/index.ts       # Interfaces TypeScript
57 | | -- tailwind.config.ts   # Design system tokens beel.*
58 | | -- package.json
59 |
60 | -- beel_logo_green_sand.png
61 | -- beel_logo_white_green.png
62 | -- beel_logo_black_sand.png
63 | -- beel_logo_black_white.png
64 | -- beel_icon_app.png
65 | -- beel_symbol_app.png

```

Chapter 5

Backend — Núcleo del Sistema

5.1 Punto de Entrada: `main.py`

El archivo `backend/app/main.py` inicializa la aplicación FastAPI con:

- **Lifespan:** conecta la base de datos al iniciar; cierra conexiones al apagar.
- **CORSMiddleware:** permite `localhost:3000`, `beel.mx`, `www.beel.mx`.
- **SlowAPIMiddleware:** rate limiting 60 req/min por IP.
- **Request ID:** header `X-Request-ID` en cada respuesta.
- **Sentry:** monitoreo de errores, muestreo de trazas al 20%.
- **7 routers** registrados bajo el prefijo `/api/v1`.

5.2 Configuración: `core/config.py`

Usa **Pydantic Settings** con lectura automática de `.env`. La instancia `settings` es global y se cachea con `@lru_cache`.

5.2.1 Variables de Entorno Completas

Variable	Default	Descripción
<code>APP_NAME</code>	<code>Beel</code>	Nombre de la app
<code>APP_VERSION</code>	<code>0.1.0</code>	Versión semver
<code>DEBUG</code>	<code>False</code>	Modo debug
<code>ENVIRONMENT</code>	<code>development</code>	dev / staging / prod
<code>DATABASE_URL</code>	<code>requerida</code>	PostgreSQL async URL
<code>REDIS_URL</code>	<code>redis://localhost:6379</code>	URL de Redis
<code>CACHE_TTL_SEARCH</code>	<code>300</code>	TTL búsquedas (5 min)

Variable	Default	Descripción
CACHE_TTL_PROPERTY	120	TTL propiedades (2 min)
CACHE_TTL_USER_PROFILE	600	TTL perfiles (10 min)
CACHE_TTL_RANKING	3600	TTL ranking (1 hora)
CLERK_SECRET_KEY	requerida	Clave secreta Clerk
CLERK_PUBLISHABLE_KEY	requerida	Clave pública Clerk
CLERK_WEBHOOK_SECRET	requerida	HMAC para webhooks
AWS_ACCESS_KEY_ID	requerida	Credencial AWS
AWS_SECRET_ACCESS_KEY	requerida	Credencial AWS
AWS_REGION	us-east-1	Región S3
S3_BUCKET_NAME	beel-media	Bucket de fotos
S3_BUCKET_URL	vacío	CloudFront CDN URL
MAX_PHOTO_SIZE_BYTES	10485760	Máx. 10 MB por foto
GOOGLE_MAPS_API_KEY	requerida	Geocodificación
LOCATION_OBFUSCATION_RADIUS	150	Radio ofuscación (m)
MERCADOPAGO_ACCESS_TOKEN	requerida	Token MercadoPago
MERCADOPAGO_PUBLIC_KEY	requerida	Clave pública MP
MERCADOPAGO_WEBHOOK_SECRET	requerida	HMAC-SHA256 MP
PLATFORM_FEE_PERCENTAGE	0.0	Comisión Beel (0% MVP)
WHATSAPP_API_KEY	requerida	API 360dialog
EMAIL_PROVIDER	resend	resend o sendgrid
EMAIL_API_KEY	requerida	Clave proveedor email
EMAIL_FROM_ADDRESS	hola@beel.mx	Remitente
SENTRY_DSN	vacío	DSN de Sentry
ALLOWED_ORIGINS	localhost + beel.mx	Orígenes CORS
RATE_LIMIT_PER_MINUTE	60	Límite general
RATE_LIMIT_AUTH_PER_MINUTE	10	Límite en auth
AVAILABILITY_HORIZON_DAYS	365	Días hacia adelante
RESERVATION_REQUEST_TIMEOUT	24	Horas para responder
PAYOUT_DELAY_HOURS	24	Demora pago al host
REVIEW_WINDOW_DAYS	7	Días para dejar reseña

5.3 Autenticación: `core/auth.py`

Beel usa **Clerk** para autenticación. El backend verifica JWT RS256:

1. Descarga JWKS desde `https://{clerk_domain}/.well-known/jwks.json` (cacheado en memoria).
2. Valida firma, expiración y claims con `python-jose`.
3. En caso de error, refresca JWKS una vez (soporte a rotación de llaves).
4. Expone:
 - `get_current_user` → levanta 401 si no hay token válido (`CurrentUser`)
 - `get_optional_user` → retorna None si no hay token (`OptionalUser`)

El dominio de Clerk se extrae automáticamente del `CLERK_PUBLISHABLE_KEY` mediante decodificación base64 URL.

5.4 Cache: `core/cache.py`

Patrón **cache-aside** con Redis async. Todas las operaciones fallan silenciosamente si Redis no está disponible.

- `get_cached(key)` → valor cacheado o None
- `set_cached(key, value, ttl)` → guarda con TTL en segundos
- `invalidate(pattern)` → elimina claves por patrón glob

Prefijos de claves: `beel:property:{id}`, `beel:search:{params}`, `beel:user:{clerk_id}`, `beel:ranking:scores`.

Chapter 6

Módulos del Backend

Cada módulo sigue la misma estructura: `models.py`, `schemas.py`, `service.py`, `router.py`.

6.1 Módulo: Users

6.1.1 Modelo de Datos

Campo	Tipo	Descripción
<code>id</code>	UUID (PK)	Identificador interno
<code>clerk_id</code>	VARCHAR(255)	Vínculo con Clerk (UNIQUE)
<code>email</code>	VARCHAR(255)	Email verificado (UNIQUE)
<code>phone</code>	VARCHAR(30)	Teléfono opcional
<code>full_name</code>	VARCHAR(255)	Nombre completo
<code>avatar_url</code>	TEXT	Foto de perfil
<code>role</code>	VARCHAR(20)	guest / host / admin
<code>is_phone_verified</code>	BOOLEAN	Verificación telefónica
<code>is_identity_verified</code>	BOOLEAN	Verificación de identidad
<code>host_since</code>	TIMESTAMPTZ	Fecha de conversión a host
<code>total_listings</code>	INTEGER	Propiedades publicadas
<code>total_trips</code>	INTEGER	Viajes como huésped
<code>deleted_at</code>	TIMESTAMPTZ	Soft delete

6.1.2 Endpoints — Users

GET <code>/me</code>	Perfil completo del usuario autenticado
PATCH <code>/me</code>	Actualizar nombre, teléfono, avatar
POST <code>/me/become-host</code>	Convertirse en anfitrión (acepta términos)
GET <code>/ {user_id}</code>	Perfil público de un usuario

6.2 Módulo: Properties

6.2.1 Modelo de Datos

Campo	Tipo	Descripción
id	UUID (PK)	Identificador
host_id	UUID (FK)	Anfitrión
title	VARCHAR(255)	Título del listado
description	TEXT	Descripción larga
property_type	VARCHAR(50)	casa, depto, cabaña, villa, habitacion, hostel, otro
status	VARCHAR(30)	pending_review, active, inactive, suspended, deleted
address	VARCHAR(500)	Dirección completa
neighborhood	VARCHAR(255)	Colonia
city	VARCHAR(100)	Default: Mérida
latitude / longitude	NUMERIC(10,8)	Coords exactas (privadas)
latitude_approx	NUMERIC(8,5)	Coords ofuscadas ± 150 m
max_guests	SMALLINT	Capacidad máxima
bedrooms / beds	SMALLINT	Habitaciones y camas
bathrooms	NUMERIC(3,1)	Baños (admite 1.5)
price_per_night	NUMERIC(10,2)	Precio en MXN
cleaning_fee	NUMERIC(10,2)	Tarifa de limpieza
security_deposit	NUMERIC(10,2)	Depósito de seguridad
min_stay_nights	SMALLINT	Mínimo de noches
cancellation_policy	VARCHAR(20)	flexible, moderada, estricta
instant_booking	BOOLEAN	Reserva inmediata
allows_pets / smoking / events	BOOLEAN	Políticas de la casa
avg_rating	NUMERIC(3,2)	Calificación promedio
ranking_score	NUMERIC(8,4)	Score de posicionamiento

6.2.2 Tablas Relacionadas

- **property_photos** — URL, thumbnail, orden, **is_primary**
- **amenities** — catálogo: slug, nombre ES/EN/PT, emoji, categoría
- **property_amenities** — tabla pivote (property_id, amenity_id)

6.2.3 Búsqueda de Propiedades

La búsqueda combina múltiples estrategias:

1. **Texto:** ILIKE sobre título y colonia con **unaccent**
2. **Geolocaliz.:** **earthdistance** para filtrar por radio en km
3. **Disponibilidad:** sub-query a **availability** para excluir fechas bloqueadas

4. **Filtros:** tipo, precio mín/máx, mascotas, instant_booking, huéspedes
5. **Orden:** ranking_score DESC

6.2.4 Endpoints — Properties

GET /search	Búsqueda: destino, fechas, huéspedes, tipo, precio, geo
GET /amenities	Catálogo de amenidades
GET /{property_id}	Detalle completo con fotos y host
POST /	Crear propiedad (requiere ser host)
PATCH /{property_id}	Actualizar (solo el host propietario)
DELETE /{property_id}	Soft-delete
GET /host/my-listings	Propiedades del host autenticado

6.3 Módulo: Reservations

6.3.1 Máquina de Estados

```

instant_booking=True
|
[SOLICITUD] --> pending --> confirmed --> completed
                |               |
                |---(rechazo)--> rejected
                |---(guest)---> cancelled_guest
                '---(host)----> cancelled_host

```

6.3.2 Campos Clave del Modelo

- **Snapshots de precio:** price_per_night_snapshot, cleaning_fee_snapshot, platform_fee_snapshot, total_price — inmutables al crear la reserva.
- **Snapshot de política:** cancellation_policy_snapshot
- **Tiempos límite:** host_response_deadline (24 h desde solicitud), payout_scheduled_at (24 h post check-in)
- **Anti-doble-booking:** SELECT FOR UPDATE SKIP LOCKED sobre availability

6.3.3 Tabla: Availability

Una fila por propiedad/fecha. UniqueConstraint(property_id, date). blocked_reason: reservation, host_block, maintenance. price_override: precio especial por temporada.

6.3.4 Endpoints — Reservations

GET /availability/{property_id}	Disponibilidad de un rango (público)
GET /price-breakdown	Desglose de precios sin crear reserva
POST /	Crear reserva (bloquea fechas atómicamente)
GET /my-trips	Viajes del huésped autenticado
GET /host-requests	Solicitudes recibidas por el host
GET /{reservation_id}	Detalle de una reserva
POST /{reservation_id}/respond	Host acepta o rechaza
POST /{reservation_id}/cancel	Cancelar reserva activa
POST /availability/{property_id}/block	Host bloquea fechas
POST /availability/{property_id}/unblock	Host desbloquea fechas

6.4 Módulo: Payments

6.4.1 Flujo de Pago

```
1. Guest: POST /reservations
   --> status = pending / confirmed

2. Guest: POST /payments/checkout/{id}
   --> Beel crea preferencia MP
   --> Retorna { checkout_url, sandbox_init_point }

3. Guest es redirigido a MercadoPago

4. MP: POST /payments/webhook/mercadopago (HMAC-SHA256)
   --> Beel verifica firma, consulta pago en API de MP

5. Si status = approved:
   --> Payment.status = "approved"
   --> Reservation.status = "confirmed"
   --> Bloqueo de fechas en availability
   --> payout_scheduled_at = check_in + 24h
```

6.4.2 Endpoints — Payments

POST /checkout/{reservation_id}	Inicia checkout en MercadoPago
GET /{reservation_id}	Estado del pago de una reserva
POST /webhook/mercadopago	Notificaciones MP (HMAC-SHA256)

6.5 Módulo: Messaging

6.5.1 Diseño

- Una conversación por par (guest_id, host_id, property_id) — UniqueConstraint
- Mensajes append-only — no se pueden editar, solo soft-delete (is_deleted)
- Tipos: text, system, image, reservation
- Contadores: guest_unread y host_unread en la conversación
- Preview: last_message_preview para el inbox sin JOINS adicionales

6.5.2 Server-Sent Events (SSE)

- El cliente conecta a `GET /api/v1/messaging/{conv_id}/stream`
- El servidor mantiene `_sse_listeners: dict[UUID, List[asyncio.Queue]]`
- Cada nuevo mensaje hace broadcast a todas las colas registradas
- **Keepalive:** ping cada 25 s para evitar que Nginx cierre la conexión
- **Header:** X-Accel-Buffering: no
- **Reconexiones:** el cliente usa exponential backoff (1s → 30s máx.)

6.5.3 Endpoints — Messaging

GET /	Inbox del usuario autenticado
POST /	Iniciar conversación con un host
GET /{conv_id}/messages	Mensajes (paginación por cursor before_id)
POST /{conv_id}/messages	Enviar mensaje de texto
POST /{conv_id}/read	Marcar mensajes como leídos
GET /{conv_id}/stream	SSE: stream en tiempo real

6.6 Módulo: Reviews

6.6.1 Diseño Bidireccional

- `reviewer_type = 'guest'` → huésped califica la propiedad y al host
- `reviewer_type = 'host'` → host califica al huésped

Restricciones: solo disponible si `status = completed`; ventana de 7 días; una reseña por (`reservation_id`, `reviewer_id`).

Calificaciones granulares (guest → propiedad): rating, cleanliness, communication, location, value — todas entre 1 y 5.

6.6.2 Endpoints — Reviews

<code>GET /property/{property_id}</code>	Reseñas públicas (paginadas)
<code>POST /</code>	Crear reseña post-estancia
<code>POST /{review_id}/respond</code>	El host responde públicamente

6.7 Módulo: Webhooks

El endpoint `POST /api/v1/webhooks/clerk` recibe eventos de Clerk:

- Verificación HMAC-SHA256 con `svix`, tolerancia de 5 minutos
- `user.created` → upsert del usuario en la DB
- `user.updated` → actualiza email, nombre, avatar
- `user.deleted` → soft delete (`deleted_at = now()`)

Chapter 7

Base de Datos

7.1 Extensiones PostgreSQL

Extensión	Uso
pgcrypto	gen_random_uuid() para PKs
unaccent	Búsqueda sin acentos
pg_trgm	Búsqueda por similitud de texto
cube	Dependencia de earthdistance
earthdistance	Búsqueda geoespacial por radio en km

7.2 Tablas Principales

Tabla	Propósito
users	Usuarios y perfiles (guest/host/admin)
identity_verifications	Proceso KYC manual (INE, pasaporte)
properties	Listados de hospedaje
property_photos	Fotos (S3 URLs)
amenities	Catálogo de amenidades
property_amenities	Pivote property ↔ amenity
availability	Calendario de disponibilidad
reservations	Reservas con snapshots de precio
payments	Pagos MP: preferencias, estados, auditoría
conversations	Hilos de mensajería guest-host
messages	Mensajes individuales (append-only)
reviews	Reseñas bidireccionales

7.3 Migraciones Alembic

#	Descripción
001	Extensiones PostgreSQL + función <code>fn_update_updated_at</code>
002	Tabla <code>users</code> + índices + <code>identity_verifications</code>
003	Tablas <code>properties</code> , <code>property_photos</code> , <code>amenities</code> , <code>property_amenities</code>
004	Tablas <code>availability</code> , <code>reservations</code> , <code>payments</code>
005	Tablas <code>conversations</code> , <code>messages</code> , <code>reviews</code> , notificaciones, <code>analytics</code>
006	Seed de amenidades iniciales

7.4 Índices Clave

Índice	Propósito
<code>idx_users_clerk_id</code>	Lookup por auth en cada request
<code>idx_users_email</code>	Login y deduplicación
<code>uq_availability_property_date</code>	Anti-doble-booking
<code>idx_messages_conversation_created</code>	Regulación de mensajes
<code>uq_conversation</code>	Una conv. por (guest, host, property)
<code>uq_review_per_stay</code>	Una reseña por (reservation, reviewer)

Chapter 8

Frontend

8.1 Arquitectura Next.js 14 App Router

- **React Server Components (RSC):** homepage, búsqueda, detalle — fetch en servidor, mejor SEO, sin estado client-side.
- **Client Components** ("use client"): checkout, mensajes, reservaciones — estado e interacción compleja.

8.2 Design System — Tokens Beel

El archivo `tailwind.config.ts` define tokens en el namespace `beel.*`:

Token	Valor	Uso
<code>beel.primary</code>	<code>#1D9E75</code>	Botones, íconos, links
<code>beel.primary-dark</code>	<code>#157A5A</code>	Hover states
<code>beel.primary-light</code>	<code>#E8F7F2</code>	Fondos sutiles, badges
<code>beel.arena</code>	<code>#F1EFE8</code>	Fondos de página
<code>beel.tierra</code>	<code>#2C2C2A</code>	Texto principal
<code>beel.accent</code>	<code>#F5A623</code>	Destacados, badges

`globals.css` define clases utilitarias: `.btn`, `.btn-primary`, `.btn-accent`, `.card`, `.badge`, `.input`, `.avatar`, `.bubble`, `.bubble-guest`, `.bubble-host`, `.price-row`, `.divider`, `.skeleton`, `.empty-state`.

8.3 Páginas

Ruta	Tipo	Descripción
/	RSC	Homepage: hero, SearchBar, grid de propiedades, trust signals
/buscar	RSC	Búsqueda: SearchBar compacto sticky, filtros, resultados
/p/[id]	RSC	Detalle: galería, host, amenidades, BookingWidget
/p/[id]/reservar	Client	Checkout: desglose de precios, crear reserva, pago MP
/reservaciones	Client	Mis viajes: tabs Próximos / Pasados / Cancelados
/mensajes	Client	Inbox + chat SSE en tiempo real

8.4 Componentes

Componente	Descripción
Navbar	Sticky, logo Beel, Clerk SignedIn/SignedOut, hamburger móvil
SearchBar	Prop <code>compact</code> ; campos: destino, check-in, check-out, huéspedes
PropertyCard	Carousel de fotos, Heart favorito, badge Zap instant booking
SearchFilters	Pills de tipo con <code>useSearchParams</code>
BookingWidget	Selector de fechas, stepper, desglose de precios, CTA de reserva
AmenityList	slug → emoji, highlights primero, “Mostrar N más”

8.5 Hooks

useApi.ts — fetch autenticado: obtiene JWT con `getToken()`, agrega `Authorization: Bearer` en cada petición. Métodos: `get`, `post`, `patch`, `del`.

useSSE.ts — gestiona `EventSource`, escucha eventos `connected/message/system`, reconexión con exponential backoff hasta 30 s, limpieza al desmontar.

Chapter 9

Seguridad

9.1 Autenticación y Autorización

- JWT RS256 de Clerk verificado en cada request protegido (sin sesiones server-side).
- Rotación de llaves: JWKS se refresca automáticamente si la verificación falla.
- Roles: `guest`, `host`, `admin` — verificación en la capa service.
- Ownership: solo el dueño del recurso o un admin puede modificarlo.

9.2 Protección de Datos

- **Coordenadas ofuscadas:** offset aleatorio de 150 m; las exactas son privadas.
- **Soft deletes:** historial de reservas y pagos se preserva.
- **Snapshots de precio:** los precios de reservas pasadas no cambian.

9.3 Validación de Webhooks

- **Clerk:** HMAC-SHA256 via `svix`, tolerancia de 5 minutos.
- **MercadoPago:** HMAC-SHA256 con `MERCADOPAGO_WEBHOOK_SECRET`.

9.4 Rate Limiting y CORS

- 60 req/min general; 10 req/min en endpoints de auth (SlowAPI).
- CORS: solo orígenes `localhost:3000`, `beel.mx`, `www.beel.mx`.

Chapter 10

Flujos Principales

10.1 Registro y Onboarding

1. Usuario se registra en Clerk (email/Google/Apple).
2. Clerk dispara webhook `user.created` → Beel crea registro (`role = guest`).
3. Usuario puede buscar y reservar inmediatamente.

10.2 Conversión a Anfitrión

1. POST `/api/v1/users/me/become-host` con `accepts_terms: true`.
2. `role` cambia a `host`, se registra `host_since = now()`.
3. El host puede crear propiedades (inician en `pending_review`).
4. Admin las revisa y activa.

10.3 Flujo Completo de Reserva

1. Huésped busca en `/buscar` con destino, fechas y huéspedes.
2. Visita el detalle (`/p/{id}`).
3. BookingWidget calcula precios con GET `/price-breakdown`.
4. Huésped va a `/p/{id}/reservar`.
5. Sistema verifica disponibilidad y crea reserva (`SELECT FOR UPDATE SKIP LOCKED`).
6. Si `instant_booking=true`: reserva queda `confirmed` automáticamente.
7. Si no: reserva queda `pending`, host tiene 24 h para responder.
8. Huésped paga en MercadoPago.
9. Webhook MP confirma el pago → reserva activa, fechas bloqueadas.
10. 24 h post check-in: pago liberado al host.

10.4 Flujo de Reseña

1. Reserva cambia a **completed** tras el check-out.
2. Durante 7 días, huésped puede calificar la propiedad.
3. Durante 7 días, host puede calificar al huésped.
4. Al crear reseña de tipo **guest**, se recalculan promedios en la propiedad.
5. Host puede responder públicamente.

Chapter 11

Referencia Completa de API

Base URL: `https://beel.mx/api/v1` **Auth:** `Authorization: Bearer {clerk_jwt}`
Formato: JSON

Método	Ruta	Auth
Health		
GET	/health	No
Users		
GET	/users/me	Sí
PATCH	/users/me	Sí
POST	/users/me/become-host	Sí
GET	/users/{user_id}	No
Properties		
GET	/properties/search	No
GET	/properties/amenities	No
GET	/properties/{property_id}	No
POST	/properties	Sí (host)
PATCH	/properties/{property_id}	Sí (propietario)
DELETE	/properties/{property_id}	Sí (propietario)
GET	/properties/host/my-listings	Sí (host)
Reservations		
GET	/reservations/availability/{property_id}	No
GET	/reservations/price-breakdown	No
POST	/reservations	Sí
GET	/reservations/my-trips	Sí
GET	/reservations/host-requests	Sí (host)
GET	/reservations/{reservation_id}	Sí (participante)
POST	/reservations/{reservation_id}/respond	Sí (host)
POST	/reservations/{reservation_id}/cancel	Sí (participante)
POST	/reservations/availability/{property_id}/block	Sí (host)

Método	Ruta	Auth
POST	/reservations/availability/{property_id}/unblock	Sí (host)
Payments		
POST	/payments/checkout/{reservation_id}	Sí (guest)
GET	/payments/{reservation_id}	Sí (participante)
POST	/payments/webhook/mercadopago	Signature MP
Messaging		
GET	/messaging	Sí
POST	/messaging	Sí
GET	/messaging/{conv_id}/messages	Sí
POST	/messaging/{conv_id}/messages	Sí
POST	/messaging/{conv_id}/read	Sí
GET	/messaging/{conv_id}/stream	Sí (SSE)
Reviews		
GET	/reviews/property/{property_id}	No
POST	/reviews	Sí
POST	/reviews/{review_id}/respond	Sí (host)
Webhooks		
POST	/webhooks/clerk	Svix sig.

Chapter 12

Decisiones Técnicas Clave

12.1 Anti-doble-booking

La creación de reserva usa `SELECT FOR UPDATE SKIP LOCKED` sobre las filas de `availability`. Si alguna fecha está bloqueada, la transacción lanza 409. Si todas están disponibles, las bloquea y crea la reserva atómicamente.

12.2 Snapshots de Precio

Los campos `*_snapshot` en `reservations` congelan el precio al momento de la reserva. Si el host cambia su precio después, las reservas existentes no se ven afectadas. Fundamental para la confianza del usuario.

12.3 Mensajería Transport-Agnostic

El schema de `conversations` y `messages` no asume ningún protocolo de transporte. El SSE actual puede migrarse a WebSocket cambiando solo el router y el cliente, sin modificar el schema.

12.4 Privacidad de Coordenadas

La función `_obfuscate_coords()` agrega un offset aleatorio dentro de `LOCATION_OBFUSCATION_RADIUS` (150 m). Solo hosts y admins ven las coordenadas exactas.

12.5 Clerk como Identity Provider

Clerk gestiona el ciclo de vida de autenticación. Beel guarda un perfil extendido sincronizado vía webhook.

- Seguridad de clase empresarial sin implementarla desde cero.
- Soporte a email, Google, Apple, teléfono.
- JWT RS256 con rotación automática de llaves.

Chapter 13

Guía de Desarrollo Local

13.1 Requisitos Previos

Python 3.11+, Node.js 20+, PostgreSQL 16 con extensiones instaladas, Redis 7, cuenta de Clerk (gratuita), cuenta de MercadoPago Sandbox, AWS S3 bucket.

13.2 Setup del Backend

Listing 13.1: Iniciar el backend

```
1 cd backend
2 pip install poetry
3 poetry install
4 cp .env.example .env
5 # Editar .env con tus credenciales
6
7 createdb beel_db
8 poetry run alembic upgrade head
9 poetry run uvicorn app.main:app --reload --port 8000
```

13.3 Setup del Frontend

Listing 13.2: Iniciar el frontend

```
1 cd frontend
2 npm install
3 cp .env.local.example .env.local
4 # NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_...
5 # CLERK_SECRET_KEY=sk_test_...
6 # NEXT_PUBLIC_API_URL=http://localhost:8000
7
8 npm run dev # http://localhost:3000
```


Chapter 14

Roadmap

14.1 MVP v1 — Estado Actual

- ✓ Autenticación con Clerk (JWT RS256, webhooks)
- ✓ CRUD de propiedades con fotos y amenidades
- ✓ Búsqueda geoespacial con filtros
- ✓ Sistema de reservas con anti-doble-booking
- ✓ Integración MercadoPago (checkout + webhooks)
- ✓ Mensajería en tiempo real (SSE)
- ✓ Reseñas bidireccionales con promedios
- ✓ Frontend completo: 6 páginas + 6 componentes + 2 hooks
- ✓ Cache Redis con TTL diferenciado
- ✓ Rate limiting y monitoreo Sentry

14.2 Próximas Iteraciones

- v1.1** Notificaciones: email (Resend) + WhatsApp Business (360dialog).
- v1.2** Upload de fotos: S3 presigned URLs, compresión y thumbnails.
- v1.3** Panel de anfitrión: dashboard, calendario visual, precios por temporada.
- v1.4** Mapa interactivo: Google Maps con clustering de propiedades.
- v1.5** Verificación de identidad: KYC (INE/pasaporte + selfie).
- v2.0** Expansión regional: Valladolid, Izamal, Progreso, Holbox.
- v2.1** App móvil: React Native con Expo.
- v3.0** Comisiones: activación de `PLATFORM_FEE_PERCENTAGE`.

Apéndice A: Catálogo de Amenidades

- wifi
- piscina
- estacionamiento
- aire_acondicionado
- cocina
- lavadora
- secadora
- tv
- netflix
- gym
- jacuzzi
- terraza
- balcon
- jardin
- parrilla
- pet_friendly
- desayuno
- acceso_discapacitados
- caja_fuerte
- extintor
- botiquin
- detector_humo

Apéndice B: Tipos de Propiedad

Código	Descripción
casa	Casa completa
departamento	Apartamento
cabaña	Cabaña o chalet
villa	Villa con amenidades premium
habitacion	Habitación en casa compartida
hostal	Hostel / dormitorio
otro	Tipo especial